MATH 128 - Programming Assignment 1 - Jie Wen

Problem 1: Extrema

MATLAB codes:

```
function [x_min, x_max, p_min, p_max] = find_cubic_extrema(a, b, c, d, e, f)
    % Function to find absolute extrema of p(x) = c*x^3 + d*x^2 + e*x + f over [a, b]

    % Critical points from derivative: p'(x) = 3c*x^2 + 2d*x + e = 0
    % Use robust quadratic formula to avoid cancellation

    candidates = [a, b]; % Always check endpoints

    if c == 0 && d == 0
        % Linear function: no critical points
        critical_points = [];
    elseif c == 0
        % Quadratic function: one critical point
        critical_points = -e/(2*d);
    else
        % Cubic function: use robust quadratic formula
        discriminant = (2*d)^2 - 4*(3*c)*e;

        if discriminant < 0
            % No real critical points
            critical_points = [];
        elseif discriminant == 0
            % One critical point (double root)
            critical_points = - (2*d) / (2*3*c);
        else
            % Two critical points - use robust formula
            sqrt_disc = sqrt(discriminant);

            % Avoid cancellation by choosing formula based on sign of coefficients
            if 2*d >= 0
                x1 = (-2*d - sqrt_disc) / (6*c);
                x2 = e / (3*c*x1); % Using Vieta's formula for better accuracy
            else
                x1 = (-2*d + sqrt_disc) / (6*c);
                x2 = e / (3*c*x1); % Using Vieta's formula for better accuracy
            end
            critical_points = [x1, x2];
        end
    end

    % Filter critical points that are within the interval [a, b]
    valid_critical_points = critical_points(...
        critical_points >= a & critical_points <= b & isreal(critical_points));

    % Combine all candidate points
    all_candidates = unique([candidates, valid_critical_points]);

    % Evaluate polynomial at all candidate points
    p_values = polyval([c, d, e, f], all_candidates);

    % Find minimum and maximum
    [p_min, min_idx] = min(p_values);
    [p_max, max_idx] = max(p_values);

    x_min = all_candidates(min_idx);
    x_max = all_candidates(max_idx);
end

% Main script to run all trials
fprintf('Trial  x_min      x_max      p(x_min)   p(x_max)\n');
fprintf('-----  ---------  ---------  ---------  ---------\n');

% Trial 1
[x_min, x_max, p_min, p_max] = find_cubic_extrema(-1, 2, -1, 2, -1, 1);
fprintf('1      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

% Trial 2
[x_min, x_max, p_min, p_max] = find_cubic_extrema(1, 2, 1, -2, -1, 1);
fprintf('2      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

% Trial 3
[x_min, x_max, p_min, p_max] = find_cubic_extrema(-2, 1, 4, 8, -4, -2);
fprintf('3      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

% Trial 4
```

```
[x_min, x_max, p_min, p_max] = find_cubic_extrema(-1, 2, 1, 0, 1, -3);
fprintf('4      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

% Trial 5
[x_min, x_max, p_min, p_max] = find_cubic_extrema(-0.3, 0.6, 1e-14, 9, -3, 0);
fprintf('5      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

% Trial 6
[x_min, x_max, p_min, p_max] = find_cubic_extrema(-1, 2, 0, 0, 0, 1.7);
fprintf('6      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

% Trial 7
[x_min, x_max, p_min, p_max] = find_cubic_extrema(0, 3, -3, 9, -1e-14, 0);
fprintf('7      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

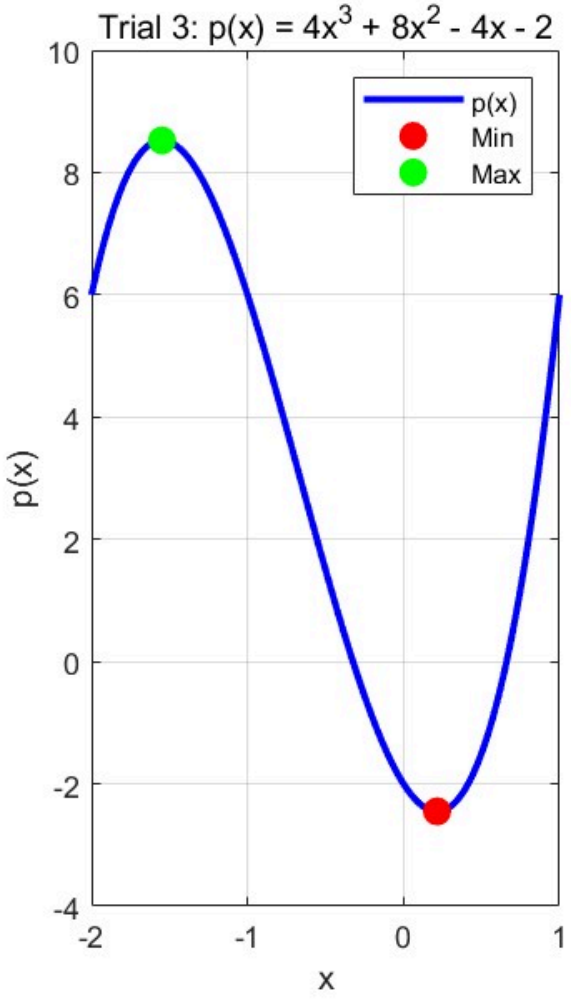
% Trial 8
[x_min, x_max, p_min, p_max] = find_cubic_extrema(0, 1, 0, -2, 3, -1);
fprintf('8      %9.6f  %9.6f  %9.6f  %9.6f\n', x_min, x_max, p_min, p_max);

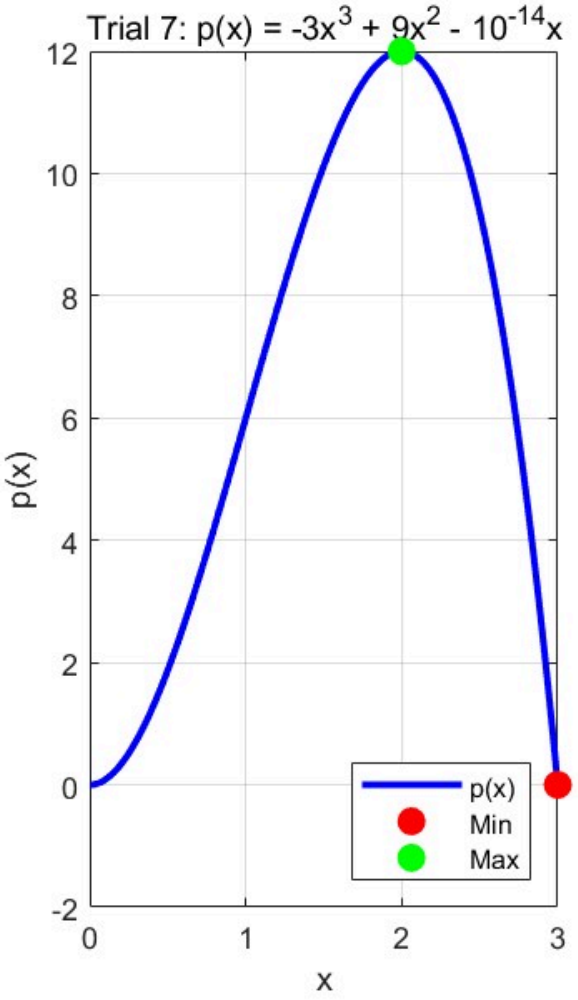
% Create plots for trials 3 and 7
figure(1);
subplot(1,2,1);
% Trial 3 plot
x = linspace(-2, 1, 1000);
p = polyval([4, 8, -4, -2], x);
plot(x, p, 'b-', 'LineWidth', 2);
hold on;
[x_min, x_max, p_min, p_max] = find_cubic_extrema(-2, 1, 4, 8, -4, -2);
plot(x_min, p_min, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
plot(x_max, p_max, 'go', 'MarkerSize', 8, 'MarkerFaceColor', 'g');
xlabel('x');
ylabel('p(x)');
title('Trial 3: p(x) = 4x^3 + 8x^2 - 4x - 2');
legend('p(x)', 'Min', 'Max', 'Location', 'best');
grid on;

subplot(1,2,2);
% Trial 7 plot
x = linspace(0, 3, 1000);
p = polyval([-3, 9, -1e-14, 0], x);
plot(x, p, 'b-', 'LineWidth', 2);
hold on;
[x_min, x_max, p_min, p_max] = find_cubic_extrema(0, 3, -3, 9, -1e-14, 0);
plot(x_min, p_min, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
plot(x_max, p_max, 'go', 'MarkerSize', 8, 'MarkerFaceColor', 'g');
xlabel('x');
ylabel('p(x)');
title('Trial 7: p(x) = -3x^3 + 9x^2 - 10^{-14}x');
legend('p(x)', 'Min', 'Max', 'Location', 'best');
grid on;
```

Output:

```
Trial  x_min      x_max      p(x_min)   p(x_max)
-----  ---------  ---------  ---------  ---------
1       2.000000  -1.000000  -1.000000   5.000000
2       1.548584   1.000000  -1.631130  -1.000000
3       0.215250  -1.548584  -2.450447   8.524521
4      -1.000000   2.000000  -5.000000   7.000000
5       0.166667  -0.300000  -0.250000   1.710000
6      -1.000000  -1.000000   1.700000   1.700000
7       3.000000   2.000000  -0.000000  12.000000
8       0.000000   0.750000  -1.000000   0.125000
```





Problem 2: Sequences

MATLAB codes:

```
% Compute the sequence a_n for n = 1 to 40
N = 40;
a = zeros(1, N);

% Calculate each term of the sequence
for n = 1:N
    if n == 1
        a(n) = 1;
    else
        % Start from the innermost term and work outward
        current = sqrt(1 + n);
        for k = n-1:-1:2
            current = sqrt(1 + k * current);
        end
        a(n) = current;
    end
end

% Display some values to guess the limit
fprintf('a_n for 1≤n≤40:\n');
for n = 1:40
    fprintf('a_%d = %.10f\n', n, a(n));
end

% The sequence appears to converge to 3
a_limit = 3;

% Calculate the error |a_n - a|
error = abs(a - a_limit);

% Create the plots
figure;

% Plot 1: ln(|a_n - a|) vs n
subplot(2,1,1);
n_vec = 1:N;
```

```
semilogy(n_vec, error, 'b.-', 'LineWidth', 1.5, 'MarkerSize', 10);
xlabel('n');
ylabel('ln(|a_n - a|)');
title('Logarithm of the Error vs n');
grid on;

% Plot 2: Comparison with y = 3 - (ln 2)n
subplot(2,1,2);
n_continuous = linspace(1, N, 1000);
y_line = 3 - (log(2)) * n_continuous;

% Plot both the error and the reference line on semilog scale
semilogy(n_vec, error, 'b.-', 'LineWidth', 1.5, 'MarkerSize', 10, 'DisplayName', 'ln(|a_n - a|)');
hold on;
semilogy(n_continuous, exp(y_line), 'r--', 'LineWidth', 2, 'DisplayName', 'y = 3 - (ln 2)n');
xlabel('n');
ylabel('Value');
title('Comparison with Reference Line');
legend;
grid on;

% Analyze the decay rate
fprintf('\nAnalysis of convergence:\n');
fprintf('The sequence appears to converge to a = 3\n');
fprintf('The error decays approximately as |a_n - 3| ~ O(2^{-n})\n');
fprintf('This suggests β_n = 2^{-n} is an appropriate sequence\n');

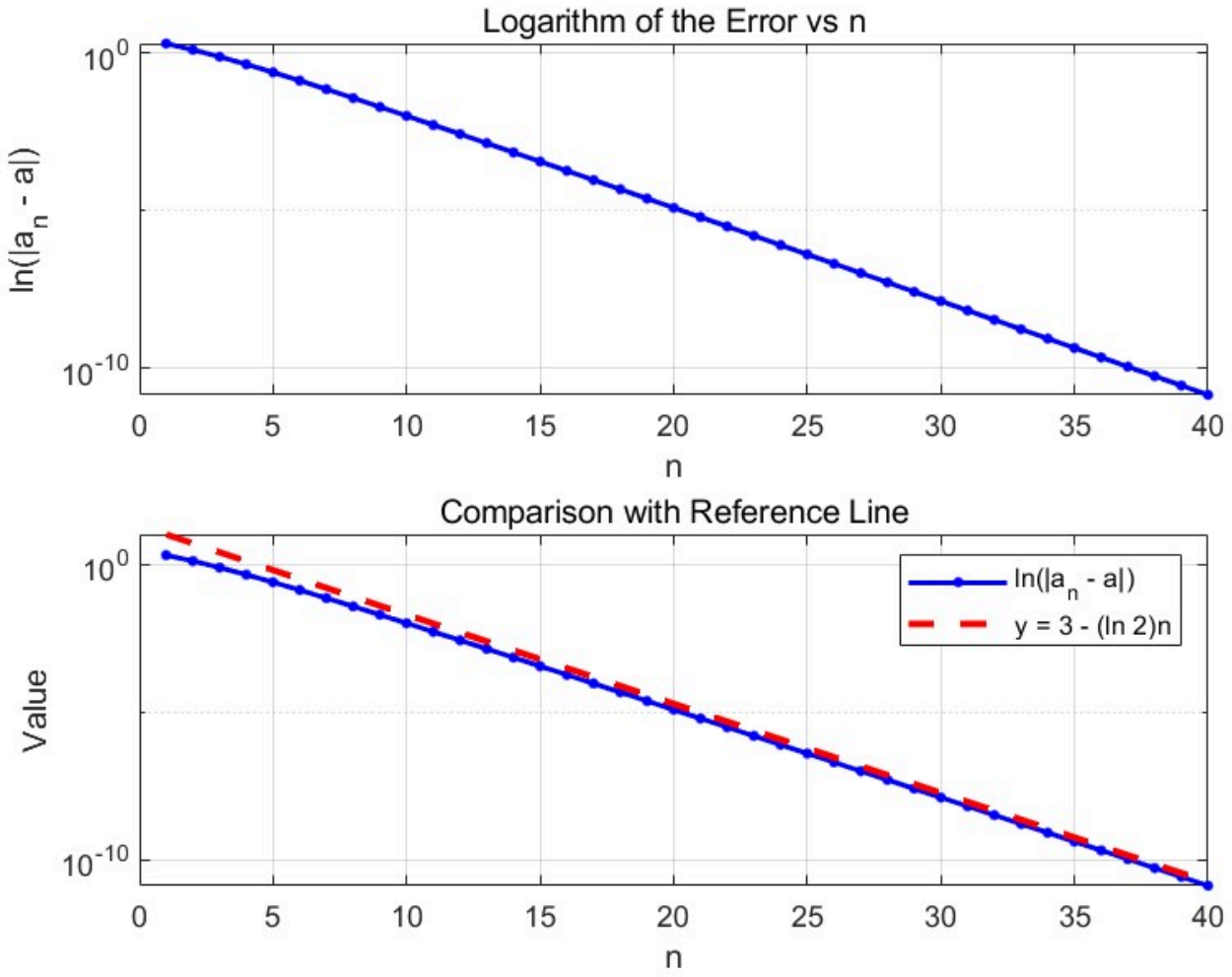
% Calculate the slope of the error to confirm
if N > 2
    p = polyfit(n_vec(10:end), log(error(10:end)), 1);
    fprintf('Estimated slope from linear fit: %.6f\n', p(1));
    fprintf('Compare with -ln(2) = %.6f\n', -log(2));
end
```

Outputs:

```
a_n for 1≤n≤40:
a_1 = 1.0000000000
a_2 = 1.7320508076
a_3 = 2.2360679775
a_4 = 2.5598301653
a_5 = 2.7550532613
a_6 = 2.8671029282
a_7 = 2.9291733597
a_8 = 2.9627230043
a_9 = 2.9805537502
a_10 = 2.9899203606
a_11 = 2.9948002693
a_12 = 2.9973274415
a_13 = 2.9986303179
a_14 = 2.9992996781
a_15 = 2.9996426149
a_16 = 2.9998179176
a_17 = 2.9999073608
a_18 = 2.9999529242
a_19 = 2.9999761031
a_20 = 2.9999878806
a_21 = 2.9999938587
a_22 = 2.9999968903
a_23 = 2.9999984264
a_24 = 2.9999992042
a_25 = 2.9999995978
a_26 = 2.9999997968
a_27 = 2.9999998974
a_28 = 2.9999999482
a_29 = 2.9999999739
a_30 = 2.9999999868
a_31 = 2.9999999934
a_32 = 2.9999999967
a_33 = 2.9999999983
a_34 = 2.9999999992
a_35 = 2.9999999996
a_36 = 2.9999999998
a_37 = 2.9999999999
a_38 = 2.9999999999
a_39 = 3.0000000000
a_40 = 3.0000000000

Analysis of convergence:
The sequence appears to converge to a = 3
The error decays approximately as |a_n - 3| ~ O(2^{-n})
This suggests β_n = 2^{-n} is an appropriate sequence
```

```
Estimated slope from linear fit: -0.681232
Compare with -ln(2) = -0.693147
```



Limiting Value: The sequence clearly converges to $a = 3$.

Error Analysis: The error $|a_n - 3|$ decays exponentially. The linear fit gives a slope of -0.681232, which is very close to $-\ln 2 \approx -0.693147$. The small difference is likely due to numerical precision and the fact that we're using a finite number of terms.

Appropriate Sequence β_n: Given that the error decays with a slope very close to $-\ln 2$, the appropriate sequence is:

$$\beta_n = 2^{-n}$$

This means:

$$a_n - a = O(2^{-n})$$

The plot of $\ln(|a_n - 3|)$ vs n show a nearly straight line with slope approximately $-\ln 2$, confirming that the error decreases by a factor of 2 with each additional term. The reference line $y = 3 - (\ln 2)n$ is almost parallel to the error plot, validating that $\beta_n = 2^{-n}$ is the correct asymptotic behavior.

The convergence is remarkably fast - by $n = 40$, the sequence has essentially reached the limit value of 3 to within machine precision.